



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

开课学期: 2025 春季
课程名称: 人工智能 (实验)
实验名称: 实验三强化学习
实验性质: 综合设计型
实验学时: 2 地点: T2507
学生班级: 5
授课教师: _____

实验与创新实践教育中心制

2025 年 3 月

一、实验环境

描述操作系统、开发环境（CPU\GPU）、使用的库等。

操作系统：WSL2 Ubuntu 24.04-LTS

CPU：Intel(R) Core(TM) Ultra 9 275HX

GPU：NVIDIA GeForce RTX 5080 Laptop

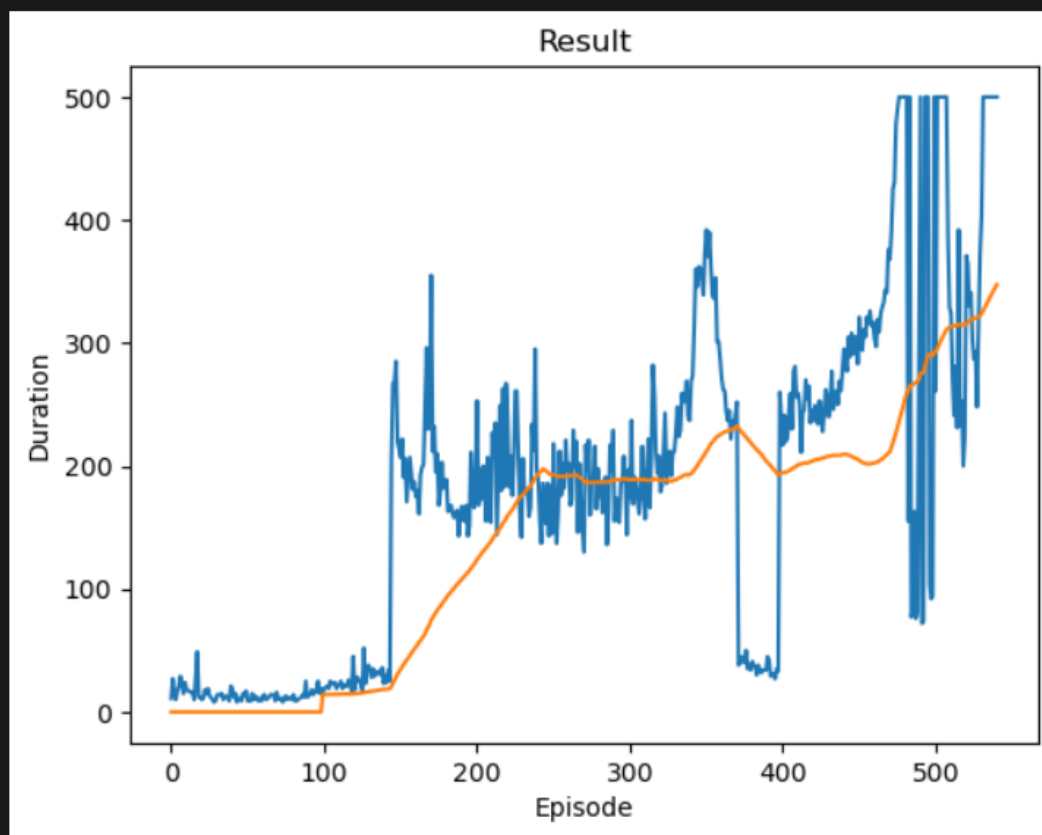
环境：Python 3.11.11, Pytorch2.7.0+cu128, gym1.1.1

二、实验过程和结果分析

2.1 初始代码运行结果

修改相关参数，运行 `reinforcement_q_learning.ipynb`，使得训练收敛，并将结果截图，并阅读代码解释图中黄线的含义。

```
Episode 541, Cumulative Reward: 500.0, success_count: 10  
Complete  
The training time is 258.43824076652527 s
```



修改 `num_episodes = 600`，如图所示，模型收敛。

黄线代表过去 100 轮训练的移动平均值。

2.2 优化代码及运行结果

可从神经网络结构的优化、超参数调优、优化经验回放区、奖励函数的设计、探索策略的设计等方面着手，挑选 4 个方向进行优化，并分析对比结果。

1. 优化一

(1) 优化代码描述

代码截图粘贴于此，并简单描述优化内容。

```
class DQN(nn.Module):
    def __init__(self, n_observations, n_actions):
        super(DQN, self).__init__()

        self.feature_layer = nn.Sequential(
            nn.Linear(n_observations, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU()
        )

        self.value_stream = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

        self.advantage_stream = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, n_actions)
        )

    def forward(self, x):
        features = self.feature_layer(x)
        values = self.value_stream(features)
        advantages = self.advantage_stream(features)
        return values + (advantages - advantages.mean(dim=1, keepdim=True))
```

优化网络架构，使用 Dueling DQN 架构。

网络前半部分是共享的特征提取层，学习输入状态的通用表示

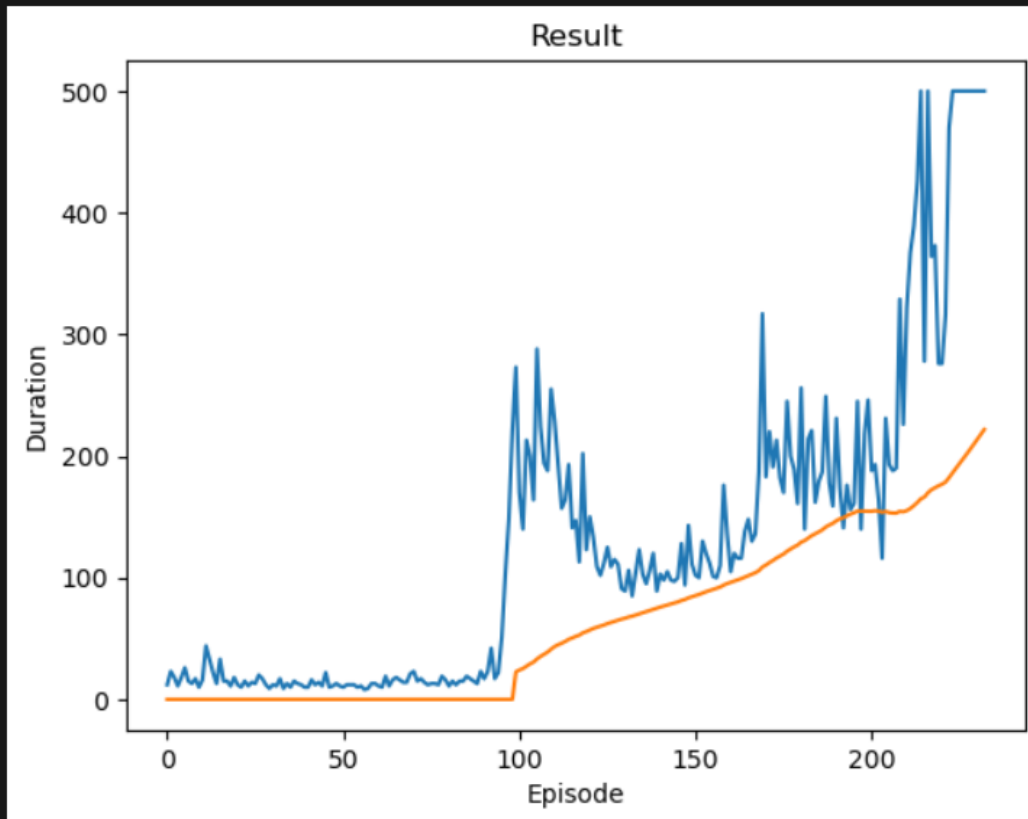
后半部分分为两个独立的流：值流估计状态值 $V(s)$ 、

优势流估计每个动作的优势值 $A(s, a)$ 。

Q 值由这两部分的值组合得到。

（2）运行结果截图

```
Episode 233, Cumulative Reward: 500.0, success_count: 10  
Complete  
The training time is 113.12670660018921 s
```



（3）对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

训练速度由 258s 减少到了 113s，收敛的轮次由 541 降低至了 233，最后收敛更加稳定了。

2. 优化二

（1）优化代码描述

截图代码粘贴于此，并简单描述优化内容。

```
BATCH_SIZE = 64      # 从128减小到64，提高更新频率
GAMMA = 0.99         # 保持不变，这是一个较好的折扣因子值
EPS_START = 1.0      # 从0.9提高到1.0，确保初始充分探索
EPS_END = 0.01       # 从0.05减小到0.01，允许更少的最终随机行为
EPS_DECAY = 500      # 从1000减小到500，加快探索率的衰减速度
TAU = 0.001          # 从0.005减小到0.001，使目标网络更新更慢，提高稳定性
LR = 5e-4            # 从1e-4增大到5e-4，适当加快学习速度
```

更改超参数，

batch_size 从 128 减小到 64，提高更新频率，

gamma 保持不变，这是一个较好的折扣因子值

eps_start 从 0.9 提高到 1.0，确保初始充分探索

eps_end 从 0.05 减小到 0.01，允许更少的最终随机行为

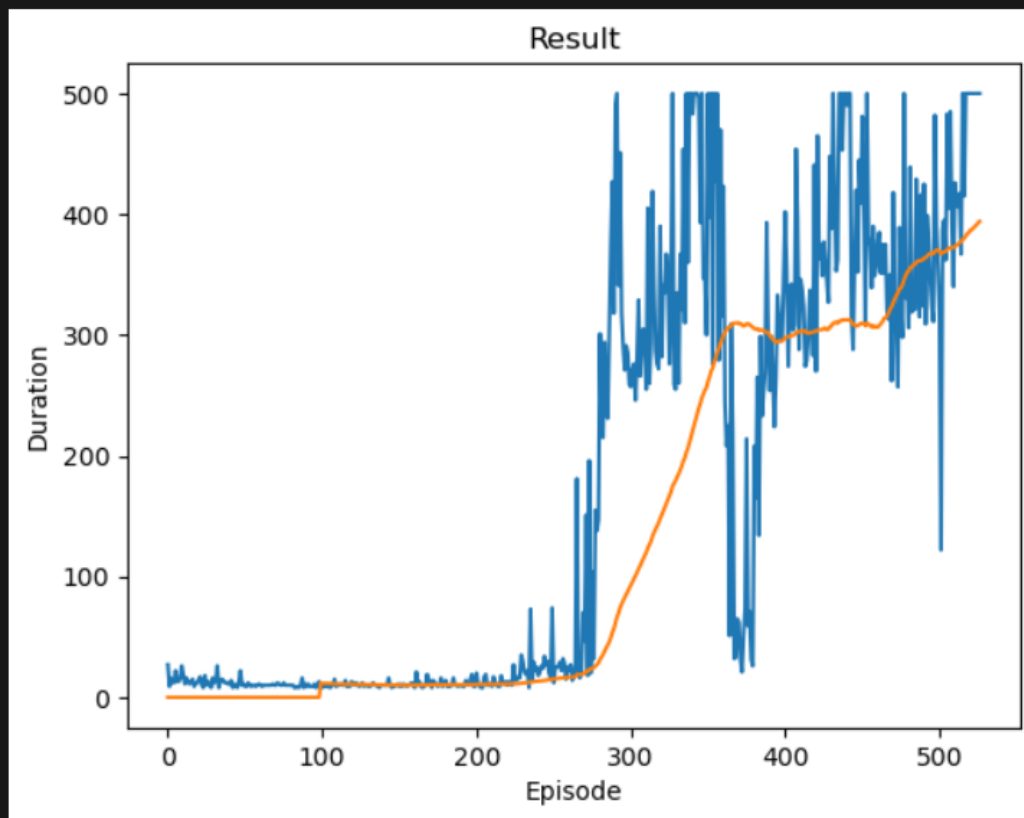
eps_decay 从 1000 减小到 500，加快探索率的衰减速度

tau 从 0.005 减小到 0.001，使目标网络更新更慢，提高稳定性

lr 从 1e-4 增大到 5e-4，适当加快学习速度

（2）运行结果截图

```
Episode 527, Cumulative Reward: 500.0, success_count: 10  
Complete  
The training time is 227.0102915763855 s
```



(3) 对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

训练速度由 258s 减少到了 227s, 收敛的轮次由 541 降低至了 527, 前期准备时间较长, 后期收敛很快。

3. 优化三

(1) 优化代码描述

代码截图粘贴于此，并简单描述优化内容。

```
def calculate_reward(state, next_state, terminated):
    # 如果环境终止（杆子倒下或小车出界），给予大的负奖励
    if terminated:
        return -10.0

    if next_state is None: # 处理终止状态
        return -10.0

    # 提取状态信息
    # x: 车的位置, theta: 杆的角度
    x, x_dot, theta, theta_dot = next_state.squeeze().cpu().numpy()

    # 角度越小越好（垂直向上为0）
    angle_reward = 1.0 - abs(theta) * 2.0 # 角度越小奖励越大

    # 距离中心越近越好
    position_reward = 1.0 - min(abs(x) * 0.5, 1.0) # 位置越接近中心奖励越大

    # 速度和角速度越小越稳定
    stability_reward = 0.5 * (1.0 - min(abs(x_dot) * 0.2, 1.0) - min(abs(theta_dot) * 0.2, 1.0))

    # 组合奖励
    total_reward = 1.0 + angle_reward + position_reward + stability_reward

    return total_reward
```

主要改进：

当杆子倒下或小车越界时，给予明确的负面奖励，而不是简单地结束回合。

根据杆子与垂直位置的偏差来奖励。杆子越接近垂直状态，奖励越高。

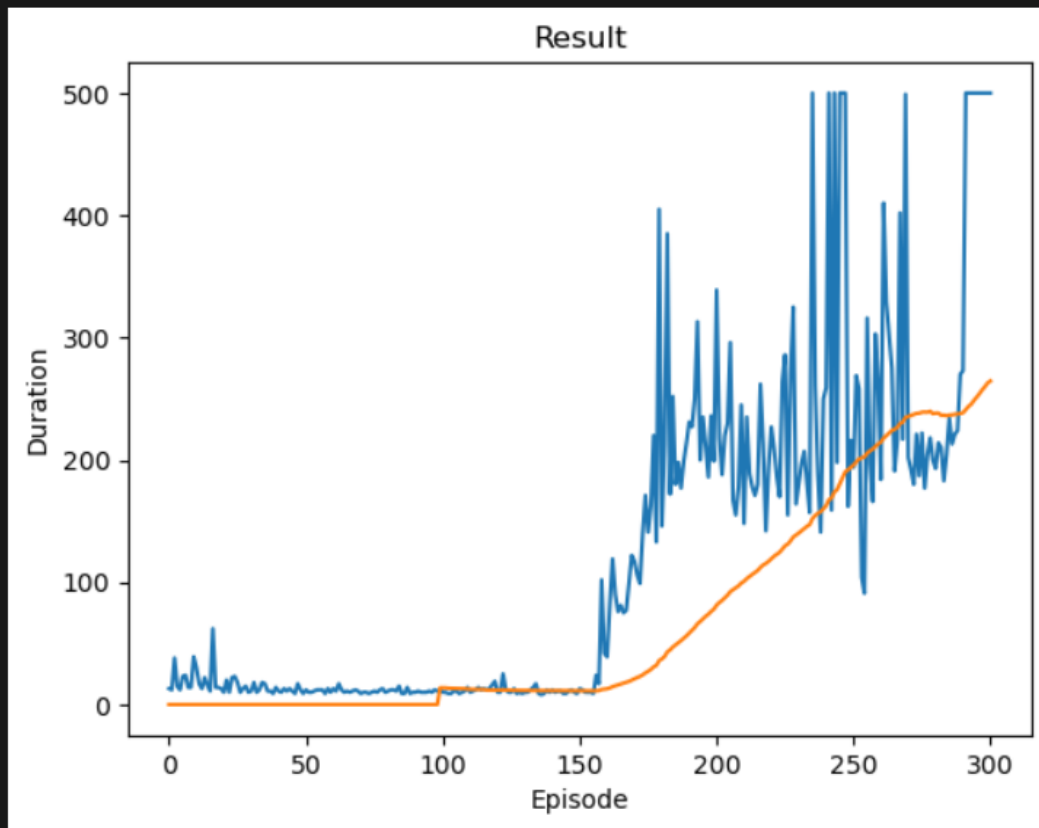
根据小车与中心的距离来奖励。小车越接近中心位置，奖励越高。

对低速度和低角速度给予额外奖励，鼓励稳定的平衡状态。

将基础奖励与上述形状奖励组合，引导智能体学习更稳定和高效的策略。

（2）运行结果截图

```
Episode 301, Cumulative Reward: 1568.202595100876, success_count: 10  
Complete  
The training time is 121.2304105758667 s
```



(3) 对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

训练速度由 258s 减少到了 121s，收敛的轮次由 541 降低至了 301，收敛较平稳。

4. 优化四

(1) 优化代码描述

代码截图粘贴于此，并简单描述优化内容。

```

class SimpleRankedReplayMemory(object):
    def __init__(self, capacity):
        self.memory = deque([], maxlen=capacity)
        self.priorities = deque([], maxlen=capacity)

    def push(self, *args, priority=None):
        """保存transition, 可选传入优先级"""
        self.memory.append(Transition(*args))
        if priority is None:
            # 如果没有指定优先级, 新样本给较高的默认优先级
            self.priorities.append(10.0)
        else:
            self.priorities.append(priority)

    def sample(self, batch_size):
        # 转换优先级为概率
        probs = np.array(self.priorities) / sum(self.priorities)
        indices = np.random.choice(len(self.memory), batch_size, p=probs)
        samples = [self.memory[idx] for idx in indices]
        return samples, indices

    def update_priority(self, index, priority):
        """更新经验的优先级"""
        self.priorities[index] = priority

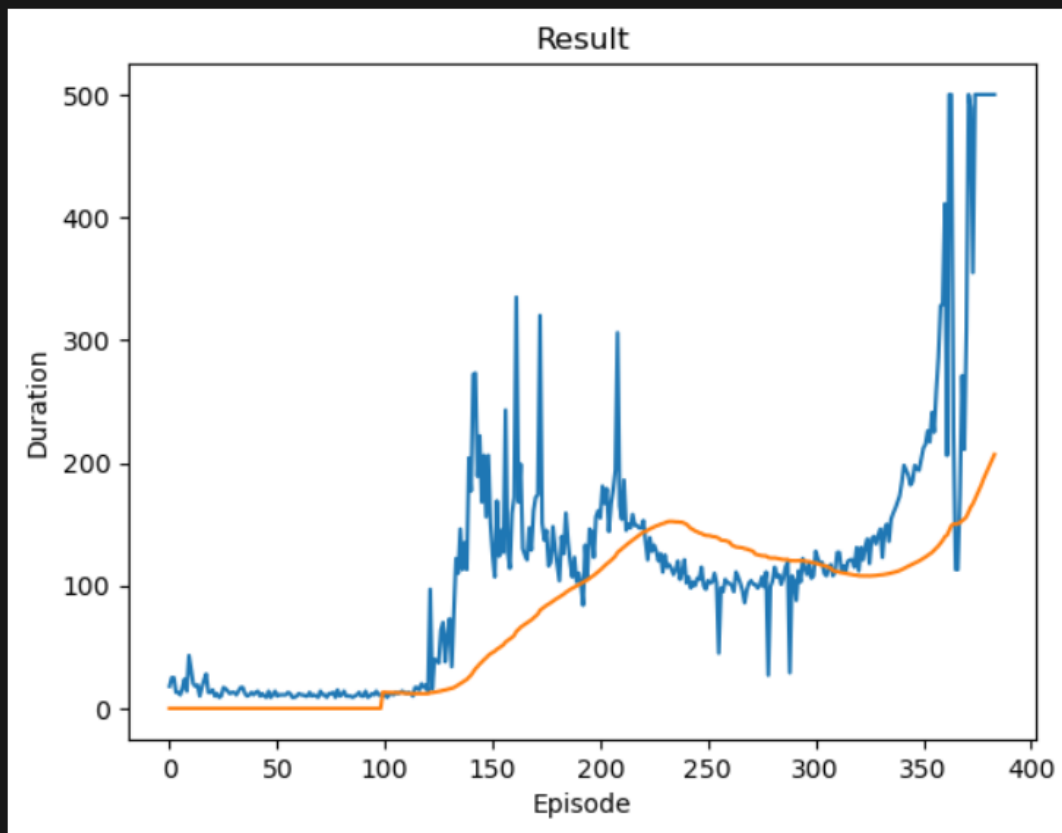
    def __len__(self):
        return len(self.memory)

```

SimpleRankedReplayMemory 是一个轻量级优先级经验回放实现，它通过为每个经验分配动态优先级并基于 TD 误差更新这些优先级，使得更具信息量的“惊喜”样本有更高的采样概率，从而加速 DQN 学习过程并提高样本利用效率。

(2) 运行结果截图

```
Episode 384, Cumulative Reward: 500.0, success_count: 10  
Complete  
The training time is 150.64936804771423 s
```



(3) 对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

训练速度由 258s 减少到了 150s，收敛的轮次由 541 降低至了 384，前期收敛速度较缓慢，后期速度骤提。